Benchmarking approximate nearest neighbour search: HNSW, IVF-Flat and IVF-PQ

CS4.406 Assignment 3

Link to Repository: github.com/Koilpillai/IRE-A3

Setup

- **Machine:** Intel i7-13650HX laptop, 10 physical cores (20 threads), 7.6 GiB RAM, WSL2 Linux. Python 3.11.7, FAISS 1.15.1, hnswlib 0.8.0, numpy 2.4.6. Everything ran on this one machine.
- **Data:** SIFT (1,000,000 × 128) and GloVe-100 (1,183,514 × 100, normalised). Euclidean distance throughout.
- **Queries:** the 10,000 queries are split in half. I pick settings on the first 5,000 and report on the last 5,000.
- **Recall@10:** my own function. An answer counts as correct if its true distance is no larger than the 10th true distance, so ties are not mistakes.
- **Speed:** queries per second (QPS), one query at a time, one thread, median of 3 timings (15 in Part 4).
- **Settings tried:** HNSW with M = 8, 16, 32 (efConstruction 200). IVF-Flat with nlist = 256, 1024, 4096. IVF-PQ with nlist 1024 and four values of m, plus nlist 4096. For each, efSearch or nprobe goes up until recall reaches 0.99 or stops rising.

Every run is a row in `results/results.csv`. `bash run_all.sh` regenerates all tables and figures.

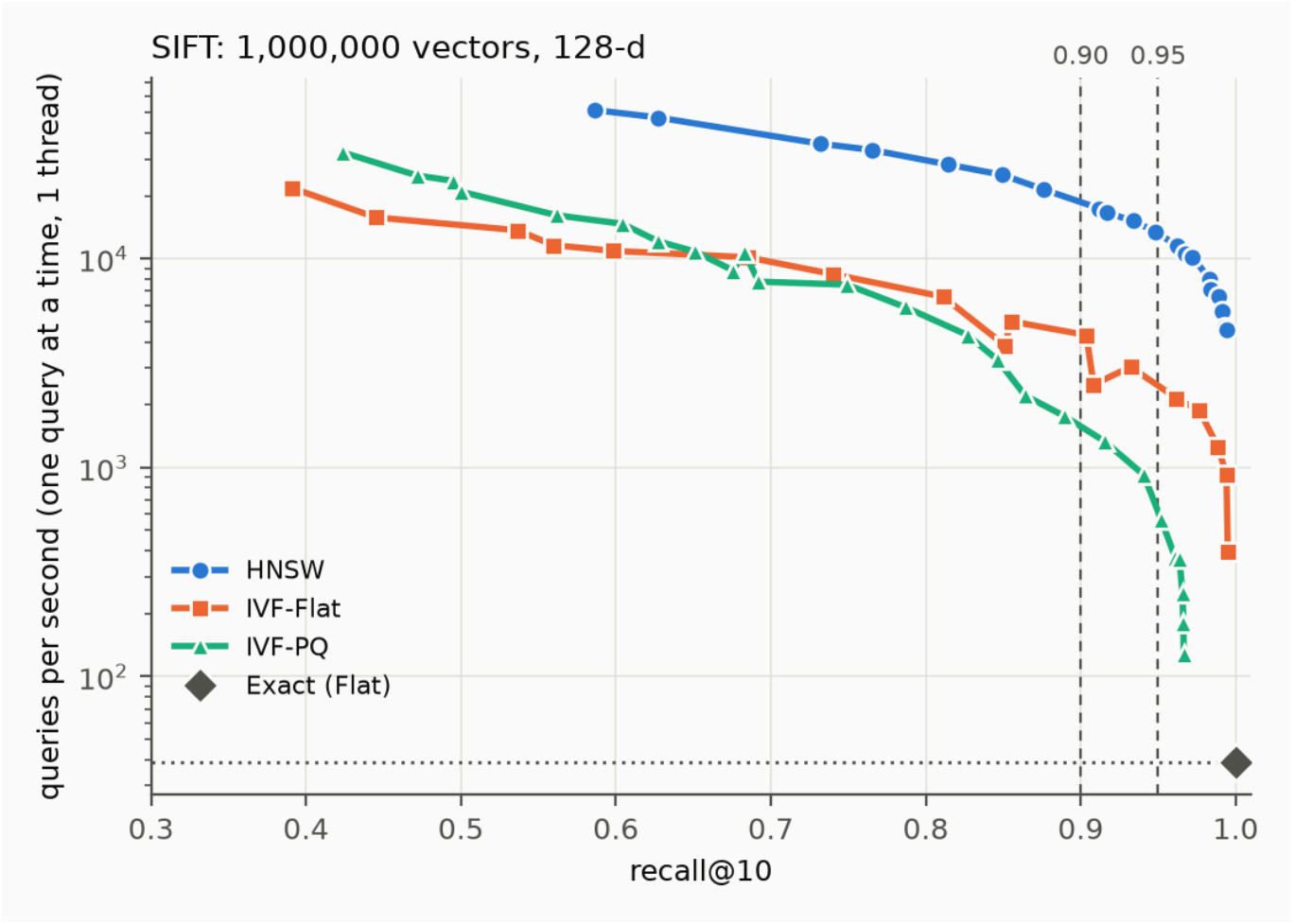
Part 1: exact search

Exact search is FAISS `IndexFlatL2`. It matches the neighbours stored in the data files (recall 1.0 both ways). A few queries (65 on SIFT, 1 on GloVe) differ only by tied distances, which is why ties have to be handled.

	one query at a time, 1 thread	one batch, 1 thread	one batch, 10 cores
SIFT	25.8 ms (39 QPS)	7.4 ms	2.8 ms
GloVe-100	23.1 ms	6.1 ms	2.7 ms

One query has to read the whole dataset (512 MB for SIFT) from RAM, so it is limited by memory speed. In a batch, blocks of vectors stay in cache while every query uses them, so each vector is read from RAM once per batch and not once per query. That makes batching about 3.5× faster on one thread, and the extra cores add about 2.5× more. I worked this explanation out from the timings; I did not measure cache misses.

Part 2: the three indexes



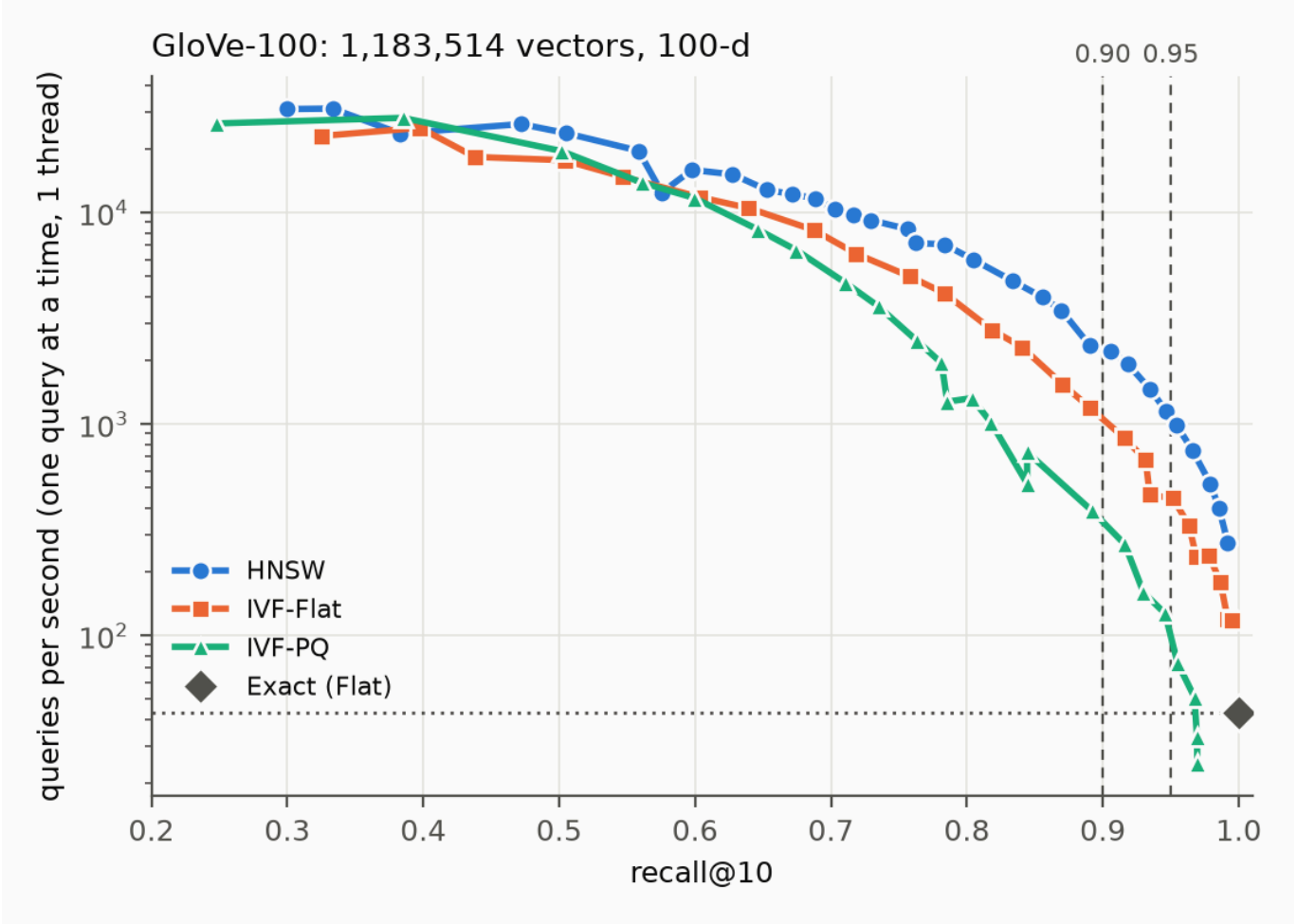


Figure 1. Recall@10 against QPS. Each line is the best result per index over all its build settings.

dataset	index	QPS at recall 0.90	QPS at recall 0.95	setting at 0.95
SIFT	Exact	39	39	–
SIFT	HNSW	16,601	11,521	M=32, efSearch=40
SIFT	IVF-Flat	3,031	2,148	nlist=4096, nprobe=48
SIFT	IVF-PQ	1,339	564	nlist=1024, m=128, nprobe=32
GloVe	Exact	43	43	–
GloVe	HNSW	2,218	984	M=32, efSearch=384
GloVe	IVF-Flat	861	445	nlist=4096, nprobe=384
GloVe	IVF-PQ	270	74	nlist=1024, m=100, nprobe=256

Table 1. Settings are picked on the tune queries; QPS is from the report queries.

Q1. At recall 0.95, which index is fastest? Does it change between datasets?

HNSW is fastest on both. What changes is the size of the lead. On SIFT it is 5.4× faster than IVF-Flat and 297× faster than exact search. On GloVe it is only 2.2× and 23×. IVF-PQ is last on both, and on GloVe it is barely faster than exact search (1.7×). GloVe is harder for every index: it needs efSearch 384 instead of 40, and nprobe 384 instead of 48.

Why. HNSW walks a graph and compares the query with only a small set of vectors. IVF has to scan every vector in the lists it opens: about 12,000 vectors on SIFT at nprobe 48, but about 110,000 (9% of the data) on GloVe at nprobe 384. So IVF loses more ground on GloVe. IVF-PQ opens the same lists as IVF-Flat, but it looks up distances in tables one byte at a time, which seems slower than IVF-Flat's plain arithmetic. I think GloVe is harder because its nearest neighbours are barely closer than other points. I did not measure either of these two explanations.

Q2. What do the indexes cost in build time and memory? What does PQ save and give up?

dataset	index	build time (s)	bytes per vector
SIFT	Exact	0.2	512
SIFT	HNSW (M=32)	86	788
SIFT	IVF-Flat (nlist 4096)	113	522
SIFT	IVF-PQ (m=128)	73	137
GloVe	Exact	0.2	400
GloVe	HNSW (M=32)	204	676
GloVe	IVF-Flat (nlist 4096)	94	409
GloVe	IVF-PQ (m=100)	64	108

Table 2. Cost of the setting that reaches recall 0.95 (builds use 10 threads).

- **HNSW** costs the most memory. It stores the vectors plus the graph links. It is also slow to build, because every insertion is a search.
- **IVF-Flat** adds almost no memory. Its build time is k-means, so it grows with nlist.
- **PQ** stores each vector as m bytes. That saves 3.8× over IVF-Flat on both datasets.
- **What PQ gives up** is accuracy. A distance from 8-bit codes has an error, and when that error is bigger than the gap between the 10th neighbour and the points behind it, true neighbours get ranked wrongly however many lists you scan. On SIFT with m=64, scanning every list gave 0.872 against 0.881 for the normal sweep, so the codes are the limit. Only m = d (one dimension per byte) reached 0.95 on both datasets. PQ is also slower (Table 1).

Part 3: compression on SIFT

Hypothesis (committed in `c8c6adc`, before any Part 3 code or result): a smaller m lowers plain recall; re-ranking helps most at small m; re-ranking costs 10–30% of speed but is faster at equal recall; fewer sub-vectors scan faster; re-ranking saves no memory.

Setup. One IVF-PQ index (nlist 1024) per m = 16, 32, 64, 128. "Plain" ranks by PQ codes only. "Re-ranked" takes the top 100 from the codes and re-ranks them with exact distances (`IndexRefineFlat`).

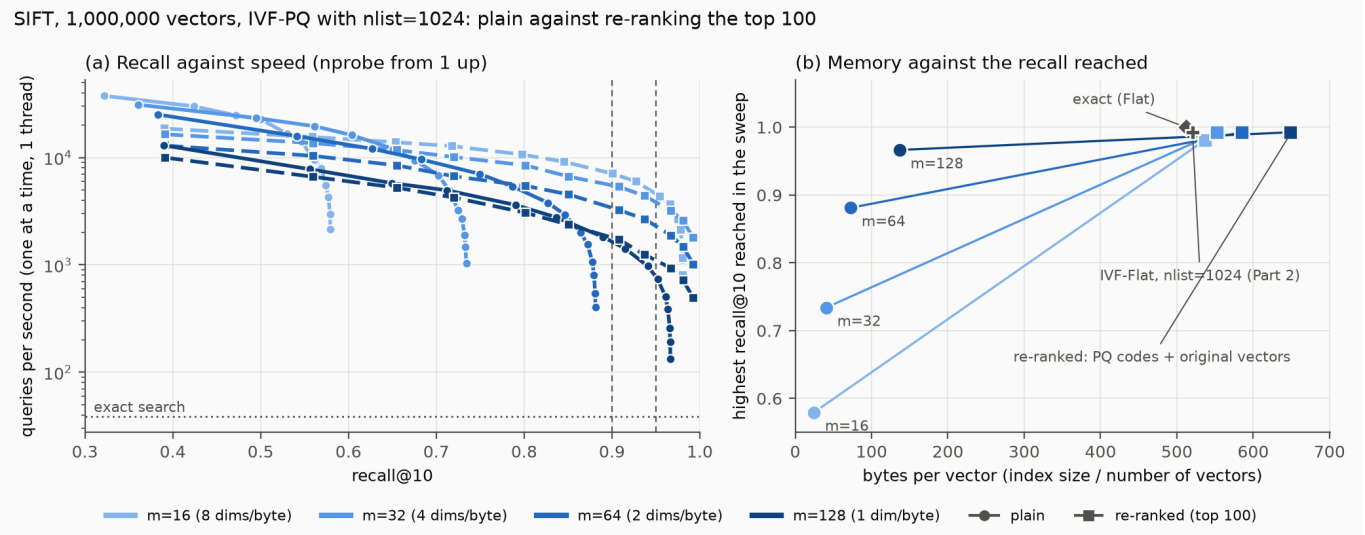


Figure 2. (a) Recall against QPS, plain (circles) and re-ranked (squares). (b) Memory against the highest recall reached.

m	version	bytes per vector	highest recall	QPS at 0.90	QPS at 0.95
16	plain	25	0.580	not reached	not reached
16	re-ranked	537	0.981	5,977	3,676
32	plain	41	0.734	not reached	not reached
32	re-ranked	553	0.992	4,383	3,201
64	plain	73	0.881	not reached	not reached
64	re-ranked	585	0.993	2,662	1,881
128	plain	137	0.967	1,415	739

m	version	bytes per vector	highest recall	QPS at 0.90	QPS at 0.95
128	re-ranked	649	0.993	1,242	930
–	IVF-Flat, nlist 1024 (Part 2)	521	0.993	2,003	1,367

Table 3. QPS is at the first nprobe that reached the target on the tune queries.

Did the results support the hypothesis? Partly: three of five predictions held.

- **Held:** plain recall falls with m (0.58, 0.73, 0.88, 0.97), and scanning every list does not help, so the codes are the limit. Plain speed roughly halves each time m doubles. Re-ranking saves no memory: 537 to 649 bytes per vector, more than IVF-Flat's 521.
- **Right direction, wrong size:** re-ranking helps most at small m, but m=16 reached 0.98, not the 0.85–0.9 I expected. Even at 8 dimensions per byte, the PQ top 100 holds 96% of the true top 10.
- **Half right:** at the same nprobe re-ranking cost a median 12–30% of speed (up to 50% at small nprobe). At equal recall it was faster only at 0.95 with m=128 (930 against 739 QPS) and 12% slower at 0.90. For smaller m, plain PQ never reaches either target.
- **Not predicted:** re-ranked m=16 is the fastest IVF at recall 0.95 (3,676 QPS against 1,367 for IVF-Flat) at about the same memory. I think it is because it reads 16 bytes per candidate instead of 512, but I did not test that.
- **Not tested:** other nlist, other re-ranking depths, GloVe, keeping vectors on disk.

Part 4: drifting data

Hypothesis (committed in `7eb8005`, before any Part 4 code or result): IVF-PQ degrades most, because its centroids and codebooks come from old data. IVF-Flat gets slower but keeps its recall. HNSW is least affected. Retraining fixes IVF, and rebuilding HNSW is the most expensive recovery. The control does not change.

What I changed. One thing: how far the data that arrives after the build is shifted. My seeded generator (`annbench/drift.py`) draws 100 Gaussian clusters in 64 dimensions. Each index is built on 200,000 old vectors. Then 100,000 new vectors are added without retraining, and 2,000 queries (1,000 to tune, 1,000 to report) are drawn like the new vectors. New vectors and queries move along one fixed direction by 0 (the control), 1, 2 or 4 times a typical vector's distance to its cluster centre. Each index keeps one search setting, chosen on the control to reach recall 0.95. I measure each index **stale** (new vectors just added) and **retrained** (rebuilt on all 300,000).

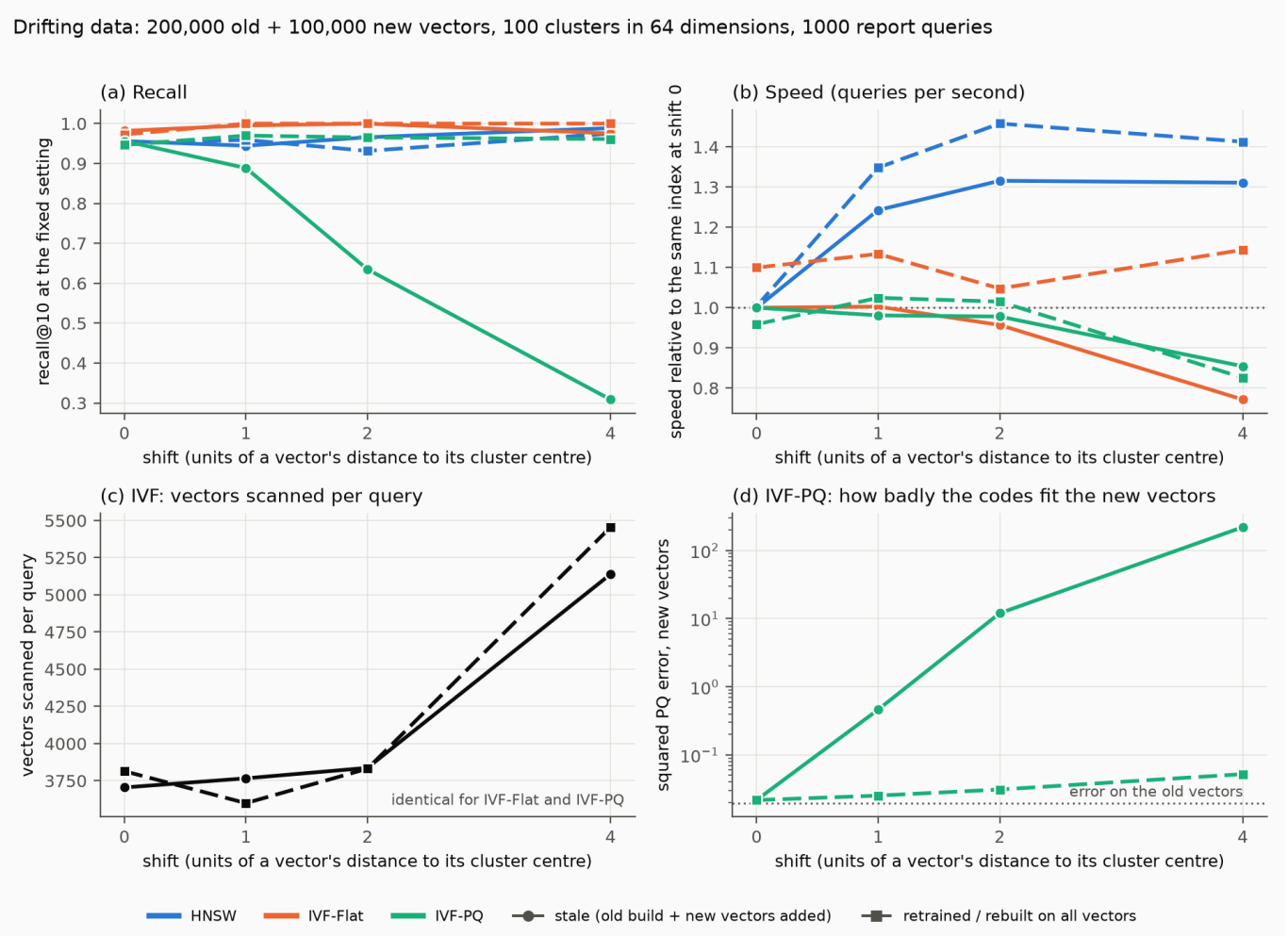


Figure 3. (a) Recall and (b) QPS at the fixed setting. (c) Vectors scanned per query by the IVF lists. (d) Squared error of the PQ codes on the new vectors. Solid = stale, dashed = retrained.

index	measure	shift 0 (control)	shift 1	shift 2	shift 4
HNSW	recall, stale	0.956	0.944	0.966	0.989
	recall, retrained	0.951	0.959	0.932	0.975
	speed, stale / retrained	1.00	0.92	0.90	0.93
IVF-Flat	recall, stale	0.982	0.995	1.000	0.975
	recall, retrained	0.973	1.000	1.000	1.000
	speed, stale / retrained	0.91	0.88	0.91	0.67
IVF-PQ	recall, stale	0.956	0.889	0.635	0.309
	recall, retrained	0.947	0.970	0.965	0.961
	speed, stale / retrained	1.04	0.96	0.96	1.03

Table 4. Recall@10, and QPS of the stale index divided by QPS of the retrained one at the same shift.

What happened.

- **IVF-PQ collapses.** Recall falls from 0.96 to 0.89, 0.64 and 0.31, and retraining brings it back to 0.96–0.97. The cause is measured: the codebook was learned on old vectors, so a shifted vector fits its code badly. The squared error of the codes on new vectors is 0.022, 0.46, 12 and 220 at shifts 0, 1, 2 and 4 (0.02–0.05 after retraining). Wrong distances put wrong vectors in the top 10, and no search setting can fix that.
- **IVF-Flat** keeps its recall, but at shift 4 it is about 30% slower than the retrained index.
- **HNSW** keeps its recall, and the stale graph is about 10% slower. I have no mechanism for this beyond the fact that nothing is trained in advance.

Supported: IVF-PQ is hit hardest and retraining fixes it. HNSW is least affected. The control stays the same.

Not supported:

- IVF-Flat did not get "several times slower". It was about 1.5× slower, at shift 4 only.
- My fat-list explanation failed. Stale and retrained indexes have similar largest lists (3,085 against 2,979) and scan similar numbers of vectors, so I do not know why stale IVF-Flat is slower.
- Rebuilding HNSW took 9.5 s, not minutes. IVF-PQ was the most expensive to retrain (22.9 s, because of its codebooks).

Not tested: gradual drift, deletions, rotations, new clusters, other sizes, real data, other nlist or M, other seeds.

Q3. What did Parts 3 and 4 show? When could the headline numbers mislead?

Part 3 showed that each compressed index's headline is only half the story. PQ's "137 bytes per vector" hides a recall that cannot go above 0.97, and below m = d it stays under 0.88. Re-ranking restores recall (0.98–0.99) but brings memory back to 537–649 bytes per vector. So a memory number needs the recall it was reached at, and a recall number needs the memory that paid for it.

Part 4 showed that the numbers in Parts 2 and 3 assume later vectors and queries look like the data the index was built on. When they do not, a trained index can fail badly (IVF-PQ at 0.31 recall with the setting that gave 0.95 before) while its speed looks normal, so nothing warns you.

The headline numbers can mislead when:

1. the data drifts after the build;
2. only one of recall, speed and memory is quoted;
3. speed is one query at a time on one thread, which says nothing about batched use (exact search gains 3.5× from batching alone);
4. settings were tuned on the same queries they are reported on. That is why I report on a second half.

Q4. 150,000 vectors, or 10 million; 100 ms per query; 16 GB RAM. Would I use ANN?

I assume SIFT-like vectors and recall 0.95. I did not time other sizes. I scale my 1M timings (25.8 ms per query on SIFT).

150,000 vectors: no ANN, use exact search. It should take about 3.9 ms per query, far under the budget, with recall 1.0, 77 MB of memory, no build and nothing to tune. An ANN index would only add tuning and a

recall below 1.

10 million vectors: use ANN. Exact search would take about 260 ms per query, 2.6 times over the budget. The raw vectors (5.1 GB) fit in 16 GB. I would use **HNSW with M=32 and efConstruction 200**, which needs about 7.9 GB at 788 bytes per vector. I would start from efSearch 40 on SIFT-like data, and about 400 on GloVe-like data. The real cost is the build: 66–86 s per million vectors, so at least 11–15 minutes, probably more. If build time or graph memory is a problem, IVF-Flat (nlist about 16,384) is the simpler second choice. I would not use plain IVF-PQ: with 16 GB there is nothing to compress, and it was the slowest index.

None of the 10M figures were measured. My laptop has 7.6 GiB of RAM, so I could not build 10M. They extrapolate from 1M, and efSearch must be re-tuned at the real size.

Q5. Two ways my benchmark could be misleading

1. Speeds are noisy and come from one laptop. They are wall-clock times on WSL2, with 3 repeats per point and one session per experiment. *Tested, partly:* I timed the same IVF-PQ settings in Part 2 and again in Part 3. The median speed ratio is 0.98, but single points were up to 36% apart. The Part 2 point behind IVF-PQ's 564 QPS in Table 1 is one of them: Part 3 measured 739 QPS for the same setting. My two Part 4 runs also differed by up to 28%. So speed differences under about 30% should not be trusted. The ranking at recall 0.95 (HNSW first by 2.2× or more) is much bigger than this noise. *Not tested:* a second machine, and batched or multi-threaded speed for the ANN indexes.

2. One HNSW build is one sample. hnswlib builds with several threads, so two identical builds give slightly different graphs. *Tested in Part 4 only:* the same rebuilt index reached recall 0.916 in an earlier run and 0.959 in the final one (the earlier run is kept in `results/earlier_run_part4_3repeats.csv`). I did not repeat the HNSW builds in Part 2, so its HNSW numbers carry about the same uncertainty. *Not tested:* whether the order of the indexes changes with other build settings.

Two explanations above (why GloVe is harder, and why IVF-PQ is slower than IVF-Flat) are likely reasons, not findings.